

Experiment: 4

Aim: To install Docker Desktop and Docker, and use existing images from Docker Hub to create containers.

Theory

1. Docker

Docker is an open-source platform used to package, deploy, and manage applications using containers.

2. Containerization

Containerization is a lightweight virtualization technique that packages an application with its dependencies and required environment so that it can run consistently.

3. Docker Image

A Docker Image is a blueprint/template used to create containers. It contains the application and required dependencies.

4. Docker Container

A Docker Container is a running instance of a Docker image. It provides an isolated environment for running an application.

5. Docker Engine

Docker Engine is the core component of Docker. It consists of:

- Docker CLI
- Docker API
- Docker Daemon

6. Docker Hub

Docker Hub is a registry containing a large collection of Docker images. Existing images such as MongoDB, Nginx and Python can be obtained from it.

7. Docker Registry

A Docker Registry stores Docker images and allows them to be versioned and shared. Images can be public or private.

8. Dockerfile

A Dockerfile contains instructions used to build a Docker image.

9. Docker Desktop

Docker Desktop provides a desktop environment for installing and managing Docker. The reference demonstrates its installation on Windows.

Docker Container vs. Virtual Machine

Docker Container	Virtual Machine
Lightweight	Comparatively large
Fast startup	Slower startup
Shares host OS kernel	Has guest OS
Uses fewer resources	Uses more resources
Easy to scale	Comparatively less efficient

Screenshots and Descriptions

1. Check Docker Version (Verify Installation)

```
yash@YashSuthar MINGW64 ~  
$ docker --version  
Docker version 27.1.1, build 6312585
```

1. Verify Docker installation. The `docker --version` command displays Docker version 27.1.1, confirming that the Docker CLI is installed.

2. View Docker System Information

```
yash@YashSuthar MINGW64 ~  
$ docker info  
Client:  
Version:           27.1.1  
Context:           desktop-linux  
Debug Mode:       false
```

2. View Docker system information. The `docker info` output shows the client version and the desktop-linux context.

3. Pull an Image from Docker Hub (Nginx)

```
yash@YashSuthar MINGW64 ~  
$ docker pull nginx  
Using default tag: latest  
latest: Pulling from library/nginx  
a8a1c5d7e95f: Pull complete  
f18232174bc9: Pull complete  
Digest: sha256:3e2b5f93...  
Status: Downloaded newer image for nginx:latest  
docker.io/library/nginx:latest
```

3. Download the Nginx image. The `docker pull nginx` command downloads the Nginx image from Docker Hub using the default latest tag.

4. Run a Container from Nginx Image

```
yash@YashSuthar MINGW64 ~  
$ docker run -d --name mynginx -p 8080:80 nginx  
f0d8a3e9c4b7e6c1d2f4438c0b3e2a1b5d6c7e8f9a0b1c2d3e4f5a6b7c8d9e0
```

4. Create an Nginx container. The command starts an Nginx container named mynginx in detached mode and maps host port 8080 to container port 80.

5. List Running Containers

```
yash@YashSuthar MINGW64 ~  
$ docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
f0d8a3e9c4b7	nginx	"/docker-entrypoint...."	10 seconds ago	Up 8 seconds	0.0.0.0:8080->80/tcp
mynginx					

5. List running containers. The docker ps output shows that mynginx is running and displays its port mapping.

6. Verify Container is Running (Logs)

```
yash@YashSuthar MINGW64 ~  
$ docker logs mynginx  
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration  
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/  
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh  
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh  
/docker-entrypoint.sh: Configuration complete; ready for start up
```

6. Inspect Nginx startup logs. The docker logs mynginx command displays initialization messages indicating that configuration is complete and Nginx is ready for startup.

7. Pull and Run a Different Image (MongoDB)

```
yash@YashSuthar MINGW64 ~  
$ docker pull mongo  
Using default tag: latest  
latest: Pulling from library/mongo  
Digest: sha256:4c9b3d1d...  
Status: Downloaded newer image for mongo:latest  
docker.io/library/mongo:latest  
  
yash@YashSuthar MINGW64 ~  
$ docker run -d --name mymongo -p 27017:27017 mongo  
7b9c1d2e3f4a5b6c7d8e9f0a1b2c3d4e5f67890123456789abcdef0123456789ab
```

7. Download and run MongoDB. The commands download the mongo image and start a detached container named mymongo, mapping host port 27017 to container port 27017.

8. List All Containers (Nginx and MongoDB)

```
yash@YashSuthar MINGW64 ~  
$ docker ps  
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS  
NAMES  
7b9c1d2e3f4a   mongo    "docker-entrypoint.s..." 12 seconds ago Up 11 seconds  0.0.0.0:27017->27017/tcp  
mymongo  
f0d8a3e9c4b7   nginx    "/docker-entrypoint...."   2 minutes ago  Up 2 minutes  0.0.0.0:8080->80/tcp  
mynginx
```

8. View both running containers. The `docker ps` output lists the running Nginx and MongoDB containers, including their status and port mappings.

9. Stop and Remove a Container

```
yash@YashSuthar MINGW64 ~  
$ docker stop mynginx  
mynginx  
yash@YashSuthar MINGW64 ~  
$ docker rm mynginx  
mynginx
```

9. Stop and remove Nginx. The `docker stop mynginx` command stops the Nginx container, and `docker rm mynginx` removes it.

10. Confirm Container Removal

```
yash@YashSuthar MINGW64 ~  
$ docker ps  
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS  
NAMES  
7b9c1d2e3f4a   mongo    "docker-entrypoint.s..." 3 minutes ago  Up 3 minutes  0.0.0.0:27017->27017/tcp  
mymongo
```

10. Check the remaining running container. After the removal command, `docker ps` shows only `mymongo` running; `mynginx` is no longer in the running-container list.